

Z-MESH

A Zero Trust architecture proposal for ephemeral Micro-MCPs in distributed sandbox runtimes

An architectural approach for secure AI agent access to corporate tools and systems through on-demand runtimes.

Author	Rodrigo Persiani Ciscom
Contributor	Rogério Alves de Macedo
Version	0.1-en
Date	June 8, 2026
Document type	Position Paper / Architecture Proposal
Status	Initial English adaptation for technical discussion, experimental validation and industry collaboration
Original version	Portuguese v0.1, DOI: 10.5281/zenodo.20600003

Repository: <https://github.com/rodperc-bit/z-mesh-architecture>

DOI: <https://doi.org/10.5281/zenodo.20600003>

This document is an English adaptation of the original Portuguese position paper. It was adapted for international technical readers and companies working on AI agent sandboxes, runtimes, MCP tooling and AI security.

Table of Contents

Abstract

1. Introduction
2. Problem Statement
 - 2.1 MCP and the N x M integration problem
 - 2.2 Conceptual operation of MCP servers
 - 2.3 Current model: persistent MCP servers
 - 2.4 Operational and security risks of persistent MCP servers
 - 2.5 Current topology with multiple persistent MCPs
 - 2.6 Summary of the problem
3. Concept and Architectural Rationale
 - 3.1 AI agent sandbox platforms and their relationship with Z-MESH
 - 3.2 Existing centralizers and orchestrators
 - 3.3 Architectural differentiation
 - 3.4 Core hypothesis
4. Z-MESH Architecture Overview
 - 4.1 AI agent and orchestration layer
 - 4.2 Z-MESH Control Hub
 - 4.3 Policy, identity and secrets layer
 - 4.4 Distributed sandbox runtimes
 - 4.4.1 Runtime microsegmentation and traffic control
 - 4.4.2 Cold-start mitigation
 - 4.5 Ephemeral Micro-MCPs
 - 4.5.1 Just-in-time credential injection
 - 4.6 Architectural synthesis
 - 4.7 Why Z-MESH is a Zero Trust architecture
 - 4.8 Architectural advantages
5. From isolated sandboxes to a Micro-MCP governance hub
6. Experimental validation proposal
7. Limitations
8. Future work and extension toward AI-WAF architectures
9. Expected contributions
10. Conclusion
11. References

Abstract

The adoption of artificial intelligence agents in corporate environments creates a new exposure surface, especially when those agents begin interacting with internal systems, databases, APIs, document repositories and operational tools. Although the Model Context Protocol (MCP) provides a standardized model for integrating AI applications with external services, the operation of broad, persistent MCP servers distributed across the network may introduce relevant risks, including excessive permissions, credential exposure, increased attack surface, lateral movement and misuse of tools by compromised agents or flows manipulated through prompt injection.

This paper proposes Z-MESH - Zero Trust Micro-MCP Ephemeral Sandbox Hub - as a complementary Zero Trust governance architecture for controlling the lifecycle of MCP capabilities invoked by AI agents. The proposal is not intended to replace MCP, agentic frameworks or AI sandbox platforms. Instead, it introduces a security control plane designed to mediate agent access to corporate systems through ephemeral, purpose-specific Micro-MCPs executed on demand inside isolated sandbox runtimes.

In this model, AI agents do not directly access internal systems and do not connect to permanently exposed MCP servers. Instead, they request capabilities from the Z-MESH Control Hub, which authenticates the agent, evaluates policy, selects or creates the appropriate Micro-MCP, instantiates it in an isolated environment, issues short-lived credentials, restricts allowed traffic, executes the authorized action, consolidates the result, records audit evidence and destroys the runtime after execution.

The proposal reframes the operational use of MCP capabilities from persistent connectors into temporary, specialized, policy-governed and auditable executions created only for an authorized purpose. As a conceptual version, this paper presents Z-MESH as an initial architecture for technical discussion, future experimental validation and controlled laboratory evolution, with the goal of assessing its functional viability in sensitive enterprise scenarios.

Keywords: Zero Trust; MCP; Model Context Protocol; AI agents; Micro-MCP; sandbox; sandbox runtime; ephemeral runtime; AI security; DevSecAI; LLM traffic; agentic security.

1. Introduction

The evolution of AI agents expands the automation capabilities of corporate environments. Models and orchestrators can now query systems, execute tasks, manipulate data and invoke external tools with increasing autonomy. This evolution, however, shifts a significant portion of security risk toward the integration layer between AI agents and corporate systems. When agents interact with internal APIs, databases, document repositories, operational tools and critical services, the attack surface is no longer limited to the model or to the application consuming it. It also includes the connectors, protocols, credentials, permissions and execution flows used to transform an agent's intent into real actions within the enterprise environment.

Reference architectures that are shaping market adoption increasingly rely on MCP servers as connectors between AI agents and internal or external resources. While this approach standardizes and simplifies integration, it may reproduce classic security problems already observed in APIs, legacy integrations and technical accounts: authorization failures, excessive permissions, long-lived or embedded credentials, low access-control granularity, limited auditability and expanded attack surface.

These risks become particularly relevant when connectors remain continuously exposed, accumulate broad access scopes or execute actions on behalf of users, applications or processes without sufficient contextual validation. As organizations scale the use of agentic systems, multiple MCP servers may be distributed across the enterprise network, each requiring hardening, monitoring, patching, access control, credential management and auditing. The resulting architecture can increase governance burden and expand the time

window available for attackers, compromised agents or prompt-injection-driven workflows to abuse available capabilities.

Z-MESH starts from a simple premise: AI agents should not hold direct, broad or persistent access to corporate resources. Although MCP already standardizes communication between AI applications and external services, Z-MESH proposes an additional governance layer based on ephemeral execution, isolation and contextual control. In this model, purpose-specific Micro-MCPs are created only when triggered by an authorized request, inside isolated sandbox runtimes and with the minimum scope required to perform the task.

The architecture therefore shifts the role of the sandbox. It is not used only as a passive containment mechanism, but as a controlled execution environment for MCP capabilities, coordinated by a Zero Trust control hub responsible for policy evaluation, runtime creation, JIT credential issuance, traffic restriction, authorized execution, audit logging and runtime destruction after the task is completed.

This document presents Z-MESH as a complementary architecture for the MCP ecosystem, focused on reducing the exposure of persistent connectors and applying Zero Trust, least privilege, ephemerality and auditability to AI agent access to sensitive corporate systems.

2. Problem Statement

2.1 MCP and the N x M integration problem

The market has been converging toward the Model Context Protocol as an emerging standard for communication and integration between AI agents and external services. MCP is an open protocol that defines how AI applications can interact in a standardized manner with tools, services, data and workflows.

At a conceptual level, MCP follows a client-server model. AI applications, often referred to as hosts, use MCP clients to connect to MCP servers. These servers expose capabilities such as tool execution, resource access, contextual information and structured interactions.

The main driver for MCP adoption is its ability to reduce point-to-point integration complexity. In a traditional model, each AI model would require custom connectors for each internal system, creating an N x M integration relationship. With MCP, models and systems connect through a common protocol layer, conceptually reducing integration effort to N + M.

For example, an organization with three AI models and four internal systems would need twelve custom integrations in a point-to-point model. Under an MCP-based architecture, the same scenario can be simplified into three MCP clients and four MCP servers, resulting in seven integration boundaries. At larger scale, the reduction becomes more significant. A scenario with ten models and fifty systems drops from five hundred point-to-point integrations to sixty protocol-level integration boundaries.

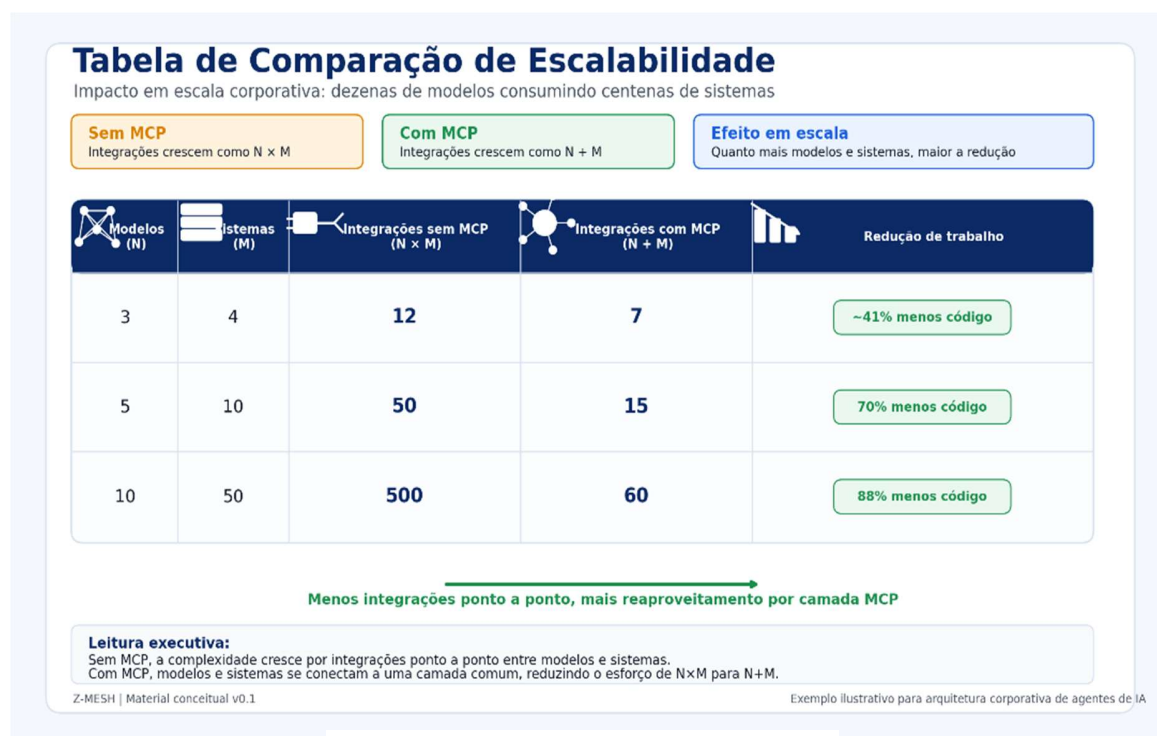


Figura 1 – Comparação de escalabilidade MCP

Figure 1 - Illustrative scalability comparison between $N \times M$ integrations and $N + M$ integrations with MCP.

This simplification improves developer productivity and architectural reuse. However, it also concentrates important security questions around the operational model used to expose MCP capabilities inside the enterprise network.

2.2 Conceptual operation of MCP servers

Structurally, an MCP server implements the protocol and exposes capabilities to AI agents through three primary primitives:

- **Tools:** executable functions that an agent can invoke dynamically, such as database queries, API calls or actions in operational systems.
- **Resources:** textual or binary data made available for agent consumption, such as logs, audit records, document repositories or operational contexts.
- **Prompts:** structured context templates or interaction patterns that guide the agent during task execution.

These servers act as connectors between agents and internal or external systems. They may be deployed locally or remotely and are responsible for intermediating communication with APIs, databases, legacy applications, corporate services and operational tools.

2.3 Current model: persistent MCP servers

Before introducing Z-MESH, it is important to clarify the predominant operational model in which MCP servers are treated as persistent connectors between agents and corporate resources. In this arrangement, MCP

servers remain available on the network to receive calls, expose tools and intermediate access to destination services.

Although this model standardizes integration and enables agent adoption at scale, it may result in broad, continuously active connectors. Because these static instances may concentrate multiple technical permissions and remain reachable for long periods, they can expand the organizational attack surface, make least privilege harder to enforce and increase potential blast radius in cases of misuse, semantic injection, credential compromise or authorization failure.

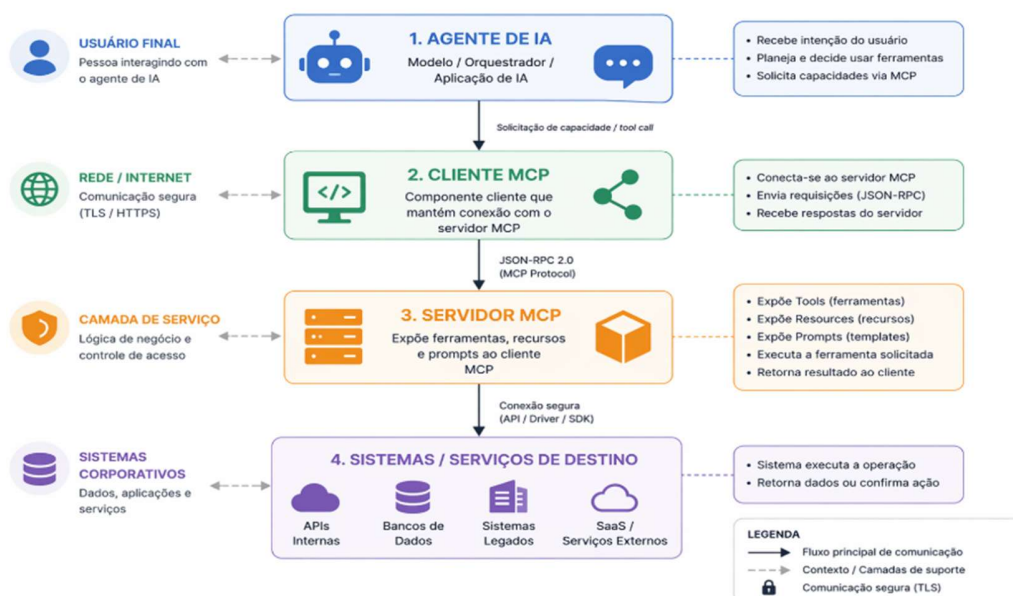


Figura 2 - Modelo atual de execução de agentes baseado em MCPs persistentes

Figure 2 - Current model of AI agents consuming persistent MCP servers.

2.4 Operational and security risks of persistent MCP servers

In current corporate environments, MCP adoption may involve multiple persistent MCP servers exposing tools, APIs, databases and internal services to AI agents. These components often operate as specialized connectors distributed across business domains such as contract management, financial data, CMDB, SIEM, cloud governance and legacy applications.

In this model, each MCP server tends to maintain continuous connectivity with destination systems. Depending on implementation choices, it may operate with pre-provisioned credentials, static access tokens or technical accounts with excessively broad scopes. As new use cases emerge, additional MCP servers are added to the infrastructure, creating a growing mesh of persistent connectors with heterogeneous control, governance and maturity levels.

Relevant risks include:

- Prolonged and static exposure of MCP servers inside the corporate network.
- Dependency on long-lived or embedded credentials associated with connectors or technical accounts.
- Expanded blast radius if a persistent MCP server is compromised.
- Potential lateral movement from improperly segmented connectors.

- Increased complexity of audit, limitation and traceability for actions executed by agents.
- Misuse of critical tools due to alignment failures, prompt injection or agent planning errors.
- Aggregation of multiple privileges in generic MCP servers, resulting in overly permissive connectors.

The core issue is not MCP as a protocol. The risk emerges from an operational model based on broad, persistent and continuously exposed connectors.

2.5 Current topology with multiple persistent MCPs

A conventional enterprise topology may include an access layer such as an AI Gateway or API Gateway, followed by AI orchestrators and a distributed mesh of persistent MCP servers. Those connectors then interact with different service domains, including databases, internal applications, APIs, document repositories and security platforms.

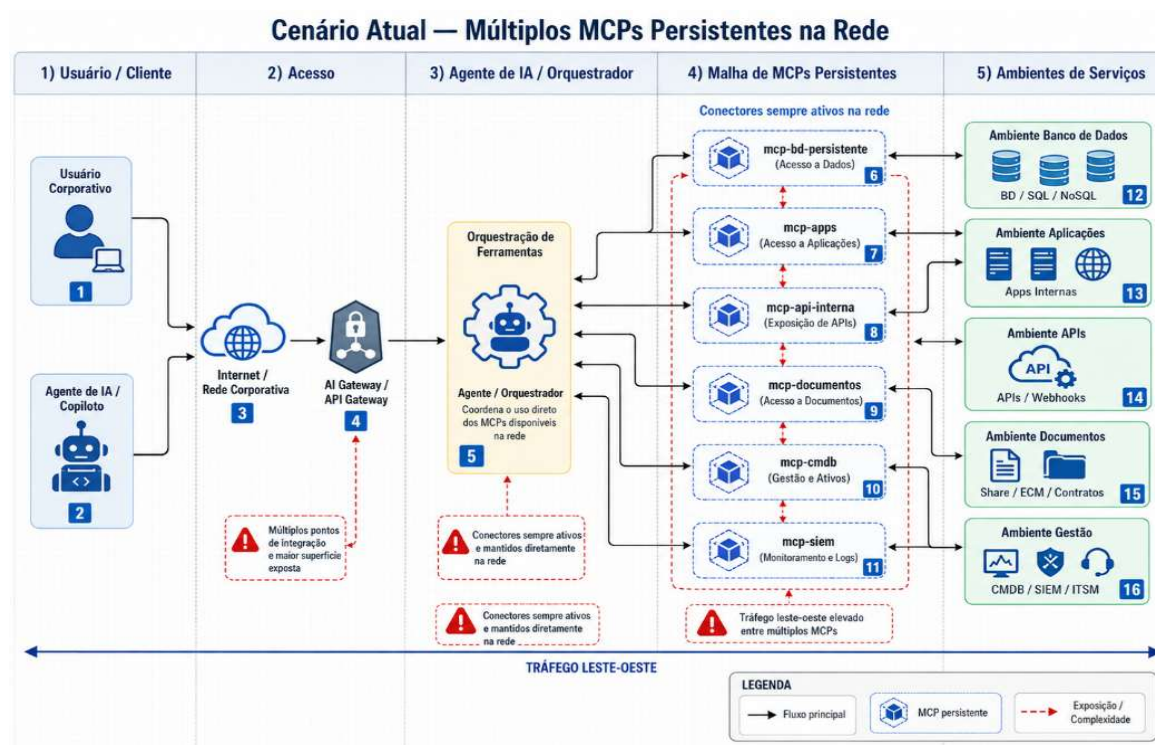


Figura 3 – Cenário Atual de múltiplos MCPs persistentes

Figure 3 - Conventional topology with multiple persistent MCP servers across the network.

This configuration increases east-west traffic, decentralizes security responsibility and expands the operational exposure surface. Each MCP server becomes a component that must be individually hardened, patched, monitored, protected, credentialed and audited. If a static connector is compromised, an attacker may obtain access to the network segment, destination systems or credentials associated with that server.

2.6 Summary of the problem

MCP is a relevant step toward standardized AI-to-system integration. However, in critical environments, its adoption requires an additional governance layer capable of enforcing isolation, scope control, temporary credentials, traffic restrictions and contextual auditability. This need motivates an ephemeral execution model

in which MCP capabilities are created only on demand, in isolated environments, under least privilege and centralized governance. This is the architectural space addressed by Z-MESH.

3. Concept and Architectural Rationale

The formulation of Z-MESH emerged from the analysis of AI agent flows in enterprise topologies, especially when agents consume tools, APIs, databases, document repositories and internal services through integration protocols such as MCP.

Traditional security sandboxes were historically designed to emulate isolated operating systems, execute suspicious binaries or analyze malicious artifacts. Their effectiveness is more limited when applied to risks specific to generative applications and autonomous agents. In these scenarios, risk is not expressed only as an executable file or conventional traffic pattern. It may also appear as semantic intent, prompt injection, tool abuse, data leakage, credential misuse and unauthorized activation of external capabilities.

At the same time, the market has developed platforms designed to provide safe execution environments for AI agents. Solutions such as E2B, microVM-based infrastructures and specialized sandbox environments demonstrate the viability of isolated and disposable runtimes for executing scripts, commands, file manipulation and tool usage in controlled environments.

The primary purpose of those platforms is to provide secure execution environments where agents can run code, commands or dynamically generated tools. Z-MESH builds on this evolution but shifts the focus. Rather than providing a general isolated machine for broad agent execution, the architecture governs the lifecycle of specific MCP capabilities created on demand, with minimum scope, temporary credentials, traffic control and destruction after execution.

The resulting question is straightforward: if lightweight virtualization and sandboxing technologies can instantiate ephemeral environments for code execution, why not apply the same logic to the communication infrastructure of AI agents by creating ephemeral, specialized, policy-governed Micro-MCPs to mediate access to sensitive corporate systems?

3.1 AI agent sandbox platforms and their relationship with Z-MESH

Platforms such as E2B represent an important evolution in the AI agent ecosystem by offering isolated environments in which agents can execute code, commands, tools and integrations inside a controlled runtime. Their primary focus is the execution plane: terminal access, filesystem operations, network access and customizable templates within a sandbox.

Z-MESH is complementary to this type of platform. While agent sandbox platforms provide isolated execution environments, Z-MESH proposes a Zero Trust control plane responsible for deciding when a capability may be invoked, which Micro-MCP must be instantiated, which temporary credential should be issued, which network destination may be reached, which data may return to the agent and when the runtime must be destroyed.

In this sense, Z-MESH does not replace AI sandbox platforms. It can use them as the execution substrate while adding governance, identity, policy enforcement, temporary credentials, restricted egress, contextual audit and post-execution teardown to the lifecycle of MCP capabilities.

3.2 Existing centralizers and orchestrators

Several frameworks can centralize or orchestrate the use of tools by AI agents, including tools exposed by MCP servers. However, those initiatives are mainly oriented toward developer experience, logical tool composition and multi-agent coordination, rather than infrastructure-level Zero Trust governance.

Microsoft Semantic Kernel

Microsoft Semantic Kernel is an SDK and framework for orchestrating AI applications and agents. It integrates models, prompts, functions, plugins and external tools. In an MCP context, Semantic Kernel can expose tools provided by MCP servers as Kernel plugins, making those capabilities available to agents and function-calling mechanisms. However, it is not primarily designed to govern the Zero Trust lifecycle of the underlying MCP servers. It does not, by itself, transform each tool into an ephemeral, sandbox-isolated Micro-MCP with JIT credentials, restricted egress, runtime audit and automatic destruction after execution.

LangChain and LangGraph

LangChain and LangGraph provide advanced capabilities for building agents, integrating tools and modeling agentic workflows as state graphs. MCP integrations can allow agents to consume tools defined by one or more MCP servers. Their main focus is logical orchestration, tool routing and workflow modeling. The secure lifecycle of MCP servers, runtime isolation, temporary credential issuance, egress control and post-execution destruction remain the responsibility of an external security architecture.

Microsoft Magentic-One / AutoGen

Magentic-One, in the AutoGen ecosystem, is a multi-agent architecture for solving complex tasks. It uses a primary Orchestrator to plan, track progress, replan and delegate subtasks to specialized agents. It should be understood as a multi-agent orchestration architecture, not as a native MCP concentrator or a Zero Trust governance layer for MCP servers. Although it distributes execution across specialized agents, it does not replace a dedicated architecture for secure MCP capability governance.

3.3 Architectural differentiation

The comparison highlights the gap Z-MESH addresses. While frameworks such as Semantic Kernel, LangChain, LangGraph and AutoGen organize the logical availability of tools to agents, Z-MESH operates on the secure lifecycle of MCP capabilities at the infrastructure level. Its contribution is not only to decide which tools are available, but also how, where, with what scope, for how long and under which policies those capabilities may exist.

By combining logical centralization, ephemeral execution, runtime isolation, temporary credentials, restricted egress and auditability, Z-MESH proposes a complementary security layer for critical corporate environments.

3.4 Core hypothesis

Z-MESH is conceived as a secure control plane for managing the lifecycle and execution of MCP capabilities under a defensive and least-privilege approach. It starts from the principle that no software component - whether autonomous agent, LLM, application or system connector - should receive implicit trust or unnecessary persistence inside the corporate network perimeter.

The central hypothesis is that the risks associated with AI agents are not inherent to MCP itself, but to an operational implementation model based on persistent, multifunctional servers with aggregated permissions. Agent security can be improved by replacing that model with ephemeral, isolated, auditable and policy-governed capabilities.

In the proposed paradigm, access to MCP tools is no longer associated with persistent servers. It becomes a temporary execution of specific capabilities. For each agent request, Z-MESH dynamically instantiates a specialized Micro-MCP, defined for a specific purpose, executed in an isolated sandbox runtime, with temporary least-privilege credentials restricted to the request context.

This changes the risk surface by reducing continuously exposed multifunctional connectors, avoiding permission accumulation in permanent technical identities and limiting the impact of abuse or compromise to a specific execution. By binding each call to an explicit identity, purpose and policy context, Z-MESH enables granular governance, contextual audit and deterministic control over agent capability usage.

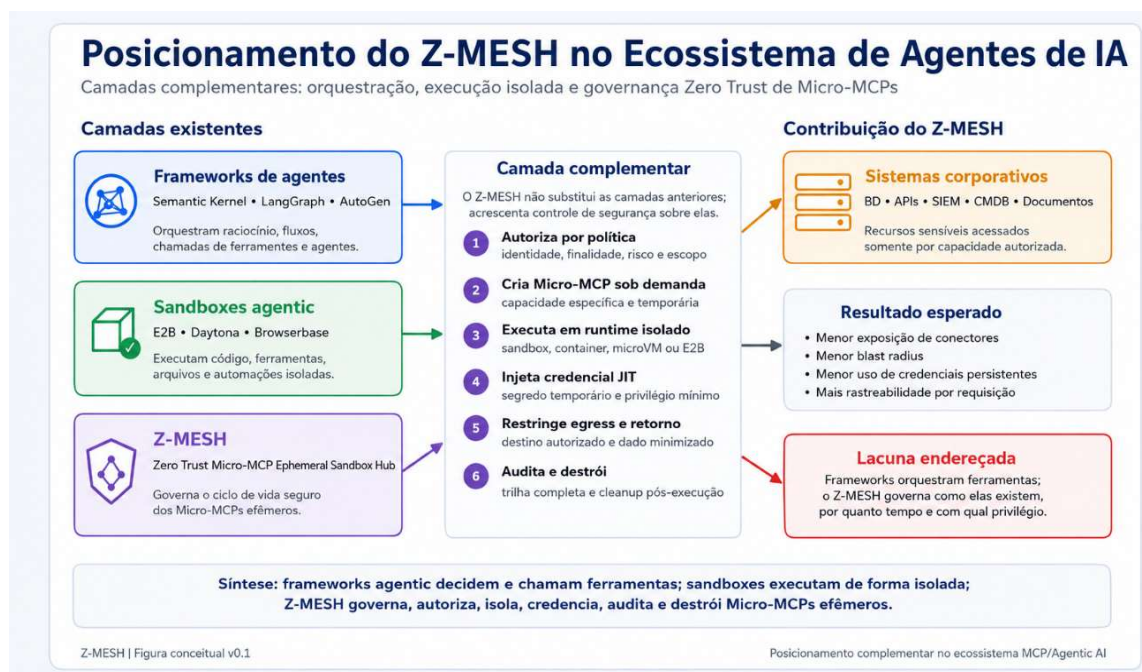


Figura 4 — Posicionamento do Z-MESH em relação a frameworks de agentes e plataformas de sandbox.

Figure 4 - Z-MESH positioning in relation to agent frameworks and AI sandbox platforms.

The agent therefore does not receive direct access to an internal system. It receives only a mediated response from an ephemeral capability created for a specific purpose, executed in an isolated environment, audited and destroyed after execution. From this premise, Z-MESH can be described as a coordinated set of layers that transform AI agent requests into controlled Micro-MCP executions with policy, identity, least privilege, temporary credentials and runtime teardown.

4. Z-MESH Architecture Overview

Z-MESH - Zero Trust Micro-MCP Ephemeral Sandbox Hub - is proposed as a secure control plane for mediating AI agent access to corporate systems through ephemeral Micro-MCPs executed in isolated sandbox runtimes and governed by policy.

Unlike the traditional model, in which MCP servers may remain continuously active in the corporate network, Z-MESH adopts a transient execution approach. The AI agent does not directly access databases, APIs, document repositories or internal services. It only sends a capability request to the Z-MESH Control Hub. From that request, the architecture evaluates identity, purpose, policy, scope and risk before instantiating the Micro-MCP required to execute the authorized task.

The architecture is composed of five main layers:

1. AI agent and orchestration layer.
2. Z-MESH Control Hub.
3. Policy, identity and secrets layer.
4. Distributed sandbox runtimes.

5. Ephemeral Micro-MCPs and corporate destination systems.

4.1 AI agent and orchestration layer

The first layer represents the consumers of the Z-MESH architecture. It includes AI agents, enterprise copilots, agentic frameworks, DevSecAI pipelines and orchestrators such as CrewAI, LangGraph, Semantic Kernel, AutoGen or equivalent solutions. These agents may interpret objectives, plan actions, select tools and request data or operations from internal systems. In Z-MESH, however, they do not receive direct access to credentials, internal addresses, databases, APIs or corporate repositories.

A logical request may look like this:

```
{
  "agent_id": "contract-analysis-agent",
  "user_id": "analyst-001",
  "purpose": "assess contract risk",
  "resource": "contracts",
  "action": "read",
  "target": "contract_001"
}
```

This approach prevents the agent from directly connecting to internal systems and enables each request to be evaluated before any real access to corporate resources is granted.

4.2 Z-MESH Control Hub

The Z-MESH Control Hub is the central logical component of the architecture. It acts as broker, control plane, operational decision point and lifecycle orchestrator for ephemeral Micro-MCPs. Its function is not only to route calls to tools, but to control whether, how, where, with what scope and for how long an MCP capability may exist.

Core responsibilities include:

- Receiving capability requests from AI agents or orchestrators.
- Authenticating the requesting agent and identifying the associated user, workload or process.
- Evaluating the declared purpose and request context.
- Consulting the policy engine.
- Selecting the appropriate Micro-MCP.
- Choosing the appropriate sandbox runtime.
- Requesting or issuing temporary credentials.
- Instantiating the Micro-MCP on demand.
- Restricting allowed traffic.
- Receiving the execution result.
- Applying data minimization, filtering or masking.
- Recording audit evidence.
- Terminating the Micro-MCP and destroying the ephemeral environment.
- Returning a consolidated and controlled response to the agent.

The Control Hub replaces direct agent access with a mediated, contextual and policy-governed model.

4.3 Policy, identity and secrets layer

The policy layer evaluates whether a request should be allowed, denied or allowed with restrictions. It functions as the decision layer of the architecture, applying access control, contextual rules, purpose validation and risk constraints.

A policy decision may consider attributes such as:

- Agent identity.
- Associated user identity.
- Call origin.
- Declared purpose.
- Data classification.
- Resource criticality.
- Requested action.
- Session risk.
- Time or operational conditions.
- Need for human approval.
- Masking or minimization requirements.
- Maximum temporary credential scope.
- Authorized runtime for execution.

An example decision could be:

```
{
  "decision": "allow_with_redaction",
  "allowed_runtime": "contracts-sandbox",
  "allowed_micro_mcp": "mcp-contracts-readonly",
  "credential_scope": "contracts.readonly",
  "credential_ttl_seconds": 120,
  "redact_fields": ["national_id", "banking_data"],
  "audit_required": true
}
```

This layer can integrate with identity providers, secret vaults, dynamic credential mechanisms, SIEM/SOC platforms and corporate governance systems. The goal is to ensure that each execution occurs under least privilege, with short-lived credentials, explicit purpose and restricted scope.

4.4 Distributed sandbox runtimes

Sandbox runtimes are isolated execution environments where Micro-MCPs are created, executed and terminated. They represent the execution plane of the Z-MESH architecture. In an initial proof of concept, these runtimes may be implemented with ephemeral containers. In higher-criticality environments, they may evolve toward stronger isolation mechanisms such as sandboxed runtimes, gVisor, Kata Containers, microVMs or Firecracker-based environments.

These runtimes can be strategically distributed across the corporate network near the data domains or services they need to access. Examples include:

- Sandbox runtime for financial data.
- Sandbox runtime for contracts.
- Sandbox runtime for internal APIs.
- Sandbox runtime for security services.
- Sandbox runtime for cloud environments.

- Sandbox runtime for lower-criticality data.

Each runtime should operate with default deny, restricted network access, no unnecessary lateral communication and automatic destruction after task execution. This layer reduces permanent connector exposure and limits the impact of improper or compromised execution because each Micro-MCP exists only for the time required to fulfill a previously authorized purpose.

4.4.1 Runtime microsegmentation and traffic control

Z-MESH assumes that each sandbox runtime operates under a default-deny network model in which no traffic is allowed without explicit authorization. Micro-MCP communication should be limited by ingress and egress policies associated with the request context, accessed data domain and purpose authorized by the policy engine.

Security therefore does not rely only on container or microVM isolation. It also requires dynamic microsegmentation of east-west traffic. Each Micro-MCP should have explicit routes and permitted destinations, communicating only with the destination system required for the authorized task. Lateral communication between Micro-MCPs should be blocked by design, preventing a compromised execution from using the runtime as a pivot point to other connectors, domains or internal systems.

4.4.2 Cold-start mitigation

The use of ephemeral Micro-MCPs introduces an operational challenge related to runtime startup time. Because the architecture creates resources on demand and destroys them after execution, cold start may become critical for user experience and low-latency workflows.

Z-MESH can mitigate this challenge through controlled pre-warming strategies such as clean runtime warm pools, microVM snapshots without sensitive context, signed images cached in advance, lightweight containers or WebAssembly-based runtimes for compatible cases. In these approaches, the base environment may remain ready for execution, but without credentials, sensitive data or request context. Identity, secrets, network policies and task parameters should be applied only after authorization by the Z-MESH Control Hub.

This separation balances performance and security. Warm-pool mechanisms should still include expiration, state cleanup, integrity verification and periodic destruction to avoid reintroducing unwanted persistence into the architecture.

4.5 Ephemeral Micro-MCPs

Micro-MCPs are minimal, specialized and disposable MCP servers. Each Micro-MCP should be designed to execute one specific capability or a very restricted set of related actions. Unlike generic, broad and continuously active MCP servers, Micro-MCPs in Z-MESH are instantiated only on demand, inside a sandbox runtime, with temporary credentials and minimum scope.

Examples include:

- mcp-contracts-readonly.
- mcp-balance-query-readonly.
- mcp-fx-rate-query.
- mcp-cmdb-readonly.
- mcp-vulnerabilities-readonly.
- mcp-siem-alerts-readonly.
- mcp-vendor-risk-query.

A Micro-MCP should not contain embedded credentials or permanent permissions. Credentials should be issued dynamically by the Control Hub, a secret vault or a workload identity mechanism, always with short time-to-live and minimal scope.

Each Micro-MCP should also define explicit rules about:

- Which actions it may execute.
- Which network destinations it may access.
- Which data it may return.
- Which fields must be masked.
- Maximum execution time.
- Required audit evidence.
- Termination conditions.

This approach turns MCP from a persistent connector into a transient, authorized, isolated and auditable capability.

4.5.1 Just-in-time credential injection

In the Z-MESH model, Micro-MCPs should not contain embedded credentials, permanent tokens or secrets stored in runtime images. Credentials should be issued only after request authorization, with short lifetime, minimum scope and explicit binding to the purpose approved by the policy engine.

Credential injection may occur during ephemeral runtime bootstrap through mechanisms controlled by the Z-MESH Control Hub, the orchestrator or an integrated secret vault. Preferably, secrets should be made available in memory, short-lived environment variables, temporary files on ephemeral volumes, dynamic tmpfs mounts or equivalent mechanisms that avoid disk persistence and reduce exposure after execution ends.

This approach does not remove the need to protect the control plane itself. The Control Hub, orchestrator, secret vault and runtime must be hardened and audited because compromise of those layers can expose credential issuance or delivery. Even so, replacing permanent secrets with temporary, contextual and disposable credentials significantly reduces exploitation windows and the impact of potential leaks.

4.6 Architectural synthesis

Z-MESH can be understood as a separation between control plane and execution plane. The control plane - represented by the Z-MESH Control Hub, policy engine, identity layer and secrets layer - decides whether a capability may be invoked, in what context, with what scope and under which restrictions. The execution plane - represented by sandbox runtimes and ephemeral Micro-MCPs - executes the authorized action in an isolated, temporary and traceable manner.

The result is a complementary layer to the MCP ecosystem that allows AI agents to consume corporate capabilities without receiving direct, broad or persistent access to internal systems.

4.7 Why Z-MESH is a Zero Trust architecture

Z-MESH can be characterized as a Zero Trust architecture because it eliminates implicit trust between AI agents, MCP connectors and corporate systems. Instead of allowing an agent to directly access persistent MCP servers or internal systems already exposed on the network, each request becomes a new security decision.

No call is considered trusted merely because it originates from an authorized agent, a known application or an internal network. Each request must be evaluated according to identity, purpose, context, scope, resource criticality, active policy and operation risk. Authorization becomes contextual and task-bound rather than static.

Zero Trust in Z-MESH is therefore expressed across the entire execution lifecycle: request validation, policy decision, credential issuance, runtime selection, traffic restriction, response minimization, contextual audit and automatic destruction of the ephemeral environment.

4.8 Architectural advantages

The primary advantage of Z-MESH is the reduction of continuous exposure associated with persistent MCP servers. In traditional architectures, every new MCP connector becomes another component requiring

protection, monitoring, patching, hardening, credential management and audit. At scale, this multiplication increases operational complexity and creates a distributed attack surface.

Z-MESH centralizes governance in a Zero Trust control plane while distributing isolated execution. The agent does not freely choose which connectors to access. It requests a capability from the Control Hub, which decides whether the action can be performed, which Micro-MCP should be created, in which sandbox runtime it should run, with which temporary credential, for how long and with which traffic and response restrictions.

Key benefits include:

- Reduced attack surface associated with continuously active connectors.
- Reduced blast radius in case of execution compromise.
- Reduction or elimination of permanent credentials in MCP connectors.
- Stronger enforcement of least privilege.
- Isolation between distinct capabilities.
- Lower lateral movement risk.
- Contextual audit by request, agent, purpose and runtime.
- Greater control over which data may return to the agent.
- Simplified governance for MCP-based tools.
- Controlled distribution of runtimes near data domains.

The result is an architecture in which integration no longer depends on static connectors, but on transient, governed and disposable capabilities.

5. From isolated sandboxes to a Micro-MCP governance hub

The traditional concept of a sandbox is associated with containing a potentially risky execution. In classic security solutions, sandboxes are used to execute files, scripts or suspicious behavior in an isolated environment to reduce impact on the primary system. In the context of AI agents and MCP, Z-MESH expands this concept.

The proposal is not merely to run a Micro-MCP inside a sandbox. The architectural contribution is to transform isolated runtimes into a governed mesh of ephemeral MCP capabilities. In this model, the Z-MESH Control Hub acts as the logical decision and governance point, while sandbox runtimes operate as distributed environments for controlled execution.

This distinction matters. Z-MESH does not need to be a single physical appliance or a single point of failure. It can be logically centralized and physically distributed, with multiple runtimes located near destination systems. What is centralized is policy, identity, decision, audit and lifecycle control. What is distributed is isolated execution according to data location, criticality and domain.

Z-MESH therefore transforms sandboxing from a passive containment mechanism into an active capability governance plane. Instead of keeping MCP servers spread across the network and permanently active, the architecture creates Micro-MCPs on demand, controls their existence, limits their privileges, records their activity and terminates the environment after the task.

This reinforces the central goal of the proposal: allowing AI agents to consume corporate capabilities without receiving direct, broad or persistent access to internal systems.

6. Experimental validation proposal

Although Z-MESH has not yet been experimentally validated in this version, an initial laboratory model is proposed to demonstrate functional viability in a controlled environment. The objective of future validation is

to demonstrate the logical flow of the architecture: access mediation by the Z-MESH Control Hub, on-demand creation of ephemeral Micro-MCPs, execution in an isolated environment, controlled response return and automatic runtime destruction after task completion.

An initial proof of concept may be built locally using low-cost and reproducible technologies such as Docker, Python and FastAPI. In this first stage, ephemeral containers would function as isolated runtimes for functional demonstration, without claiming to represent the isolation level required for high-criticality production environments.

Minimum laboratory components:

- Python-based agent simulator.
- Z-MESH Control Hub implemented with FastAPI.
- Two or three Micro-MCPs packaged as Docker containers.
- Mock APIs representing internal systems.
- Simple authorization rules.
- JSON audit logs.

The validation flow would submit an agent request to the Control Hub, evaluate applicable policy, instantiate the corresponding Micro-MCP in an ephemeral container, execute the authorized action against a simulated API, return the result to the Control Hub, record audit evidence and automatically remove the runtime after execution.

The PoC will be considered successful if it demonstrates that the agent does not directly access the mocked system, the Micro-MCP is created only on demand, execution occurs with limited scope, unauthorized requests are blocked before runtime creation, each execution generates audit evidence and the ephemeral environment is destroyed after use.

The initial laboratory is not intended to prove banking-grade production security or strong isolation equivalent to microVMs. Its initial objective is to validate the functional logic of Z-MESH. Later stages may evolve toward stronger runtimes such as gVisor, Kata Containers, Firecracker or AI sandbox platforms, and toward integration with policy engines, secret vaults, workload identity and enterprise observability.

7. Limitations

As an initial conceptual model, Z-MESH presents limitations and challenges that must be evaluated in future experimentation. Relevant points include latency impact associated with ephemeral runtime creation, operational complexity of the control plane, infrastructure cost, robust protection of the Z-MESH Control Hub, security of temporary credential issuance and the effectiveness of isolation controls across different runtime technologies.

These limitations do not invalidate the proposal. They define its scope in this initial version and indicate important points for future evolution, practical validation and architectural maturity. Z-MESH should therefore be understood as a conceptual architecture under development, whose functional viability and operational security must be progressively demonstrated through proofs of concept, controlled experimentation and evaluation in simulated enterprise scenarios.

8. Future work and extension toward AI-WAF architectures

As a possible evolution path, Z-MESH may become part of a broader protection architecture for AI-based applications, approaching the concept of a next-generation AI-WAF. In that scenario, the proposal would no longer be limited to governance of ephemeral Micro-MCPs. It would compose an integrated defense layer for traditional traffic, language model traffic and actions executed by autonomous agents.

A conventional WAF protects web applications against known attacks, HTTP request abuse, vulnerability exploitation and anomalous traffic patterns. In generative AI applications, the attack surface expands toward semantic elements such as prompt injection, jailbreaks, data leakage, context manipulation, tool misuse and unauthorized activation of external capabilities. An AI-WAF should therefore inspect not only packets, URLs or request parameters, but also intent, context, purpose, action risk and potential execution impact.

Z-MESH could contribute to this evolution by operating at the boundary between agent decision and real execution over corporate systems. While AI Gateways, LLM Firewalls or DLP layers may analyze prompts, responses and sensitive data, Z-MESH adds control over the resulting action. When an AI interaction requires access to an internal system, the architecture evaluates policy, instantiates an ephemeral Micro-MCP in a sandbox runtime, issues temporary credentials, restricts allowed traffic, minimizes the response and destroys the environment after execution.

This positions Z-MESH as a secure execution layer within a broader defense architecture capable of combining traditional traffic inspection, semantic LLM interaction analysis, tool governance, ephemeral execution, temporary credentials and contextual audit.

9. Expected contributions

The main expected contribution of Z-MESH is to propose a complementary architecture for the MCP ecosystem focused on reducing the risks associated with continuous exposure of persistent MCP servers in critical corporate environments. The proposal is not intended to replace MCP, agentic frameworks or sandbox platforms. It adds a Zero Trust governance layer for controlling the lifecycle of capabilities invoked by AI agents.

Z-MESH shifts the integration model from permanent connectors to ephemeral, specialized and auditable capabilities. Instead of maintaining broad MCP servers that are continuously active and potentially associated with persistent credentials, the architecture proposes on-demand creation of Micro-MCPs with minimum scope, isolated sandbox runtimes, temporary credentials, restricted egress and automatic destruction after execution.

Another contribution is the explicit separation between logical agent orchestration and secure execution governance. Frameworks such as Semantic Kernel, LangGraph and AutoGen can continue coordinating agents, workflows and tool calls, while Z-MESH acts as a security control plane evaluating policy, identity, purpose, scope and risk before allowing a capability to execute against sensitive corporate systems.

The proposal also introduces ephemeral Micro-MCPs as a minimum unit of secure capability. This favors least privilege, reduces blast radius, limits lateral movement, improves request-level traceability and enables stronger control over which data may return to an agent.

In summary, the expected contributions are:

- A Zero Trust architecture specific to the governance of MCP capabilities invoked by AI agents.
- Reduction of dependency on broad, persistent and continuously exposed MCP servers.
- Introduction of ephemeral Micro-MCPs as secure and specialized execution units.
- Application of temporary credentials, minimum scope, restricted egress and post-execution destruction to the tool lifecycle.
- Separation between logical agent orchestration and secure execution governance over internal systems.
- Improved auditability, traceability and contextual control of actions executed by autonomous agents.
- A conceptual foundation for future experimental validation and evolution toward AI-WAF and agentic traffic protection architectures.

10. Conclusion

The adoption of AI agents in corporate environments will continue to expand automation, system integration and the ability to execute complex tasks. However, this evolution also shifts a significant portion of risk toward the integration layer between agents, tools, APIs, databases, document repositories and internal services. MCP is an important step toward standardizing communication between AI applications and external systems, reducing point-to-point integration complexity and improving interoperability, reuse and scalability.

The central problem identified in this paper is not MCP as a protocol, but the operational model based on persistent, broad and continuously exposed MCP servers inside corporate networks. When connectors remain permanently active, aggregate multiple capabilities, use long-lived credentials or operate with excessive scopes, they become sensitive components in the AI security architecture.

Z-MESH proposes a complementary architectural approach. Its contribution is to replace persistent connectors with ephemeral, specialized and policy-governed capabilities executed in isolated sandbox runtimes. The Z-MESH Control Hub becomes responsible for mediating the relationship between AI agents and corporate systems, evaluating identity, purpose, context, policy, scope and risk before any capability is instantiated.

This change enables integrated application of Zero Trust, least privilege, ephemerality, contextual isolation, temporary credentials, egress control, data minimization and auditability across the entire lifecycle of MCP capability consumption. Instead of asking whether an agent can reach an internal system, the Z-MESH model asks whether a specific capability should exist, for a specific purpose, during a specific time window, with a specific scope and under specific controls.

This document presents Z-MESH as an initial conceptual architecture, pending experimental validation. The proposed laboratory model using ephemeral containers, Python and FastAPI offers a path to demonstrate the functional logic of the architecture before evolving toward stronger runtimes, enterprise-grade policy engines, secret vaults, workload identity and observability mechanisms.

In summary, Z-MESH aims to contribute to a safer, more controlled and auditable adoption of agentic AI in sensitive corporate environments. Its central premise is that AI agents should not directly access internal systems. They should consume governed, ephemeral, isolated and auditable capabilities created only when authorized and destroyed when no longer needed.

IT is business. IT security is the soul of the business.

11. References

- [1] ANTHROPIC. Introducing the Model Context Protocol. Anthropic, 2024.
- [2] MODEL CONTEXT PROTOCOL. Model Context Protocol Documentation and Specification. Model Context Protocol Project.
- [3] OPENAI. MCP and Connectors: Tools and Connectors Guide. OpenAI Developers Documentation.
- [4] GOOGLE CLOUD. What is Model Context Protocol? Google Cloud Documentation.
- [5] MICROSOFT. Model Context Protocol overview. Microsoft Learn.
- [6] MICROSOFT. Semantic Kernel Documentation. Microsoft Learn.
- [7] MICROSOFT. Give agents access to MCP Servers. Semantic Kernel Documentation. Microsoft Learn.
- [8] MICROSOFT. Magentic-One: A generalist multi-agent system for solving complex tasks. Microsoft Research / AutoGen Documentation.
- [9] LANGCHAIN. Model Context Protocol integration. LangChain Documentation.
- [10] LANGGRAPH. LangGraph Documentation: agents, graphs and orchestration. LangChain Documentation.

- [11] E2B. E2B Documentation: sandboxes for AI agents. E2B Documentation.
- [12] DOCKER. Docker Compose Documentation. Docker Documentation.
- [13] FIRECRACKER MICROVM. Firecracker Documentation and Project Repository. Firecracker MicroVM Project.
- [14] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. NIST SP 800-207: Zero Trust Architecture. NIST, 2020.
- [15] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. NIST SP 800-53: Security and Privacy Controls for Information Systems and Organizations. NIST.
- [16] OWASP FOUNDATION. OWASP API Security Top 10. OWASP.
- [17] OWASP FOUNDATION. OWASP Top 10 for Large Language Model Applications. OWASP GenAI Security Project.
- [18] OWASP FOUNDATION. MCP Security Cheat Sheet. OWASP Cheat Sheet Series.
- [19] OPEN POLICY AGENT. Open Policy Agent Documentation. OPA Project.
- [20] HASHICORP. Vault Documentation: static and dynamic secrets. HashiCorp Developer Documentation.
- [21] SPIFFE / SPIRE. Secure Production Identity Framework for Everyone. SPIFFE Project.
- [22] HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston: Addison-Wesley, 2003.